Building a Task That Passes

HarmonyGames · Universe date: October 1, 2025, 00:00 PT (07:00 UTC) or February 14, 2026, 00:00 UTC · Updated October 28, 2026

This guide is based on thousands of real reviews in the Eval Portal. It focuses on the mistakes that cost CBs the most time and the most tasks. Keep the one-page checklist (Appendix) open while you work.

0 · How it works

> THE CORE RULE
>
> **Hard to find, one answer once found.** A careless agent should get the task wrong. A careful agent should reach exactly one correct result.



- The reviews run in this order: **/injection → /prompt → /hardness → /oe → /rubric → submit.** PASS and NON-FAIL both count as passing. Each step below is one section of this guide, and the dark boxes above are the reviews. If a review fails, go to **section 9**.
- **Where people get stuck:** the prompt and hardness steps are where most CBs give up. The rubric step takes the most rounds, usually 4 reviews.
- **Ask for help early.** When a review fails, ask the assistant: “Help me fix the failed items in my latest /prompt report.” It works only on what failed, starting from what you meant. Use `/admin` only for platform problems, such as a run that crashed or a pass-card issue.
- **Keep moving.** Most tasks take 1–2 days from start to submit. If you go quiet for too long, your claim can lapse and the task goes to someone else.

1 · Explore the data in the Eval Portal

Find a real decision your persona would make, with the evidence spread across services.

- **Check your assigned persona and universe date first.** There are two dates: **Oct 1, 2025** and **Feb 14, 2026**. In the Feb 14 universe, the studio announced its shutdown on **Feb 11, 2026**, so business-as-usual premises are wrong there. A task built on the other universe date fails.

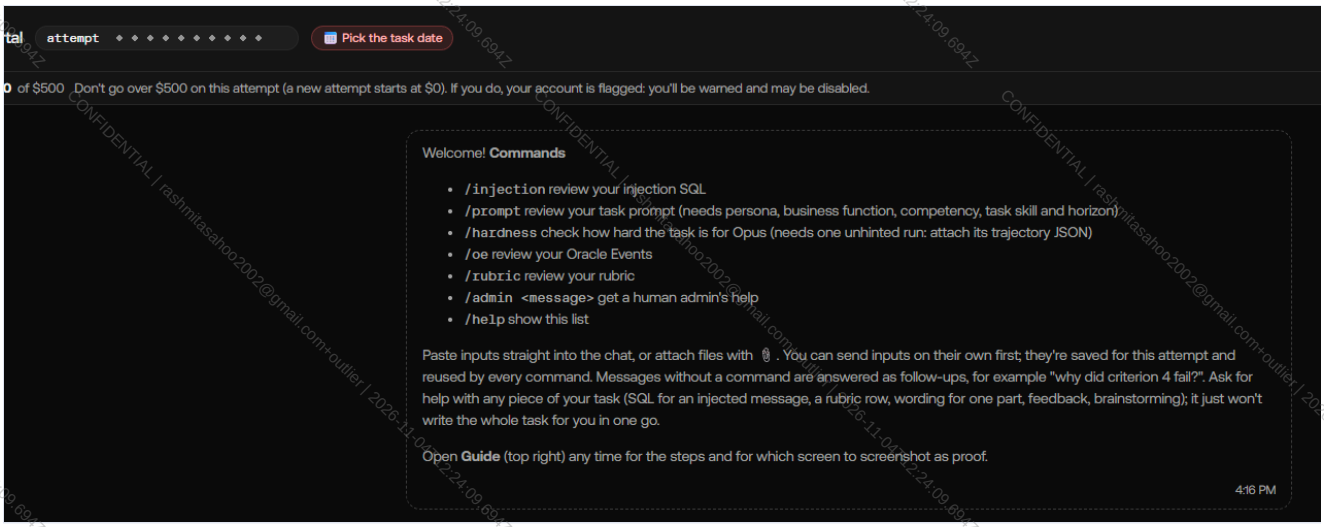


Image 1 · A new attempt: the review commands, and Pick the task date at the top.

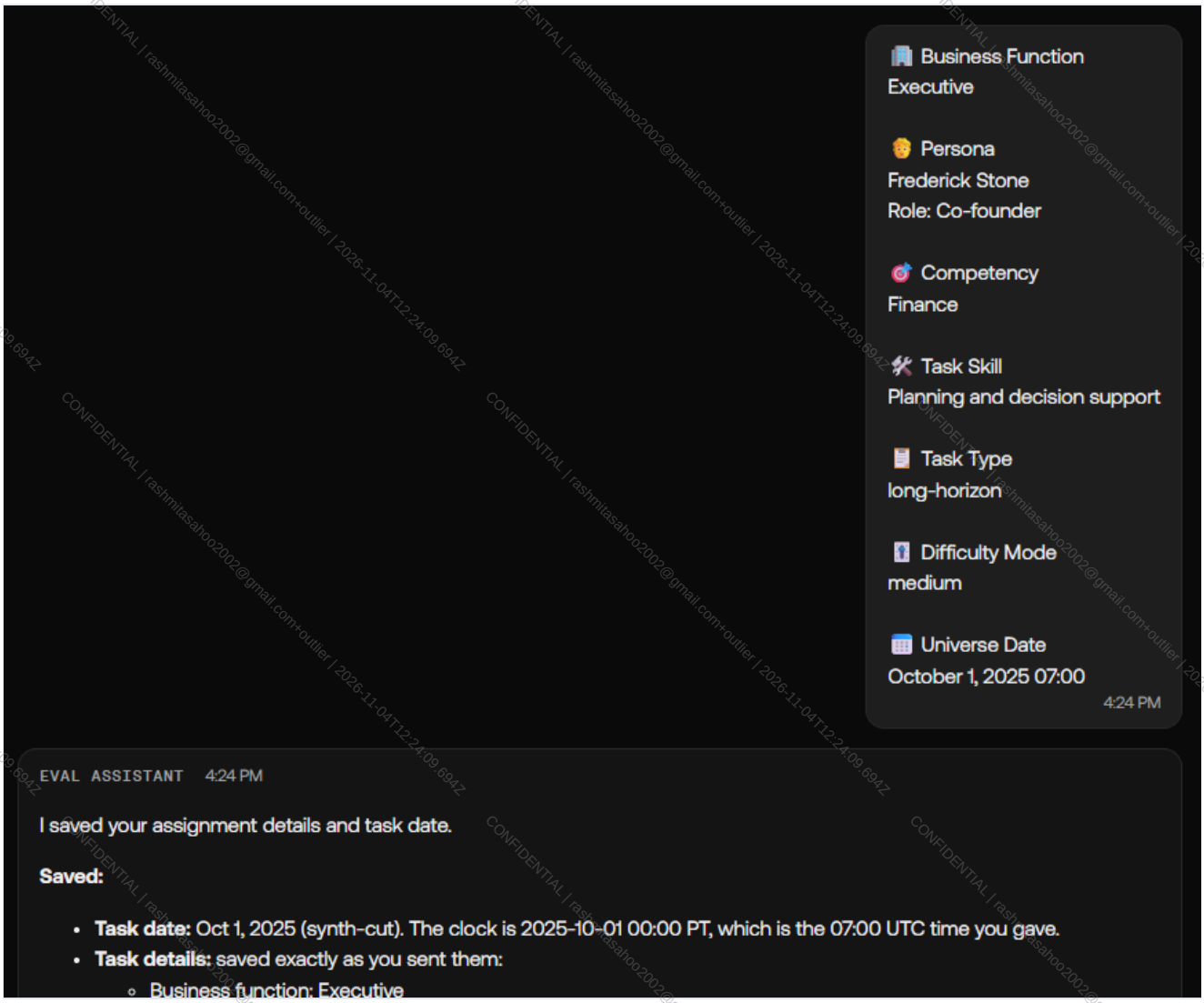


Image 2 · The assignment and the task date, saved for this attempt.

- **Ask the assistant.** It can search every service for you. Copy and paste these:

COPY AND PASTE

"What can [persona] see about [topic]? List the channel, date and author."

"Find every record across services that mentions [entity] between [dates], and flag any that disagree."

"Was [person] still employed and in [channel] on [date]?"

"Which items are borderline for [my rule]? Show the records."

"What does the base data already disagree about for [topic]?"

"Are there known base-data issues for [topic]? Check before I build on it."

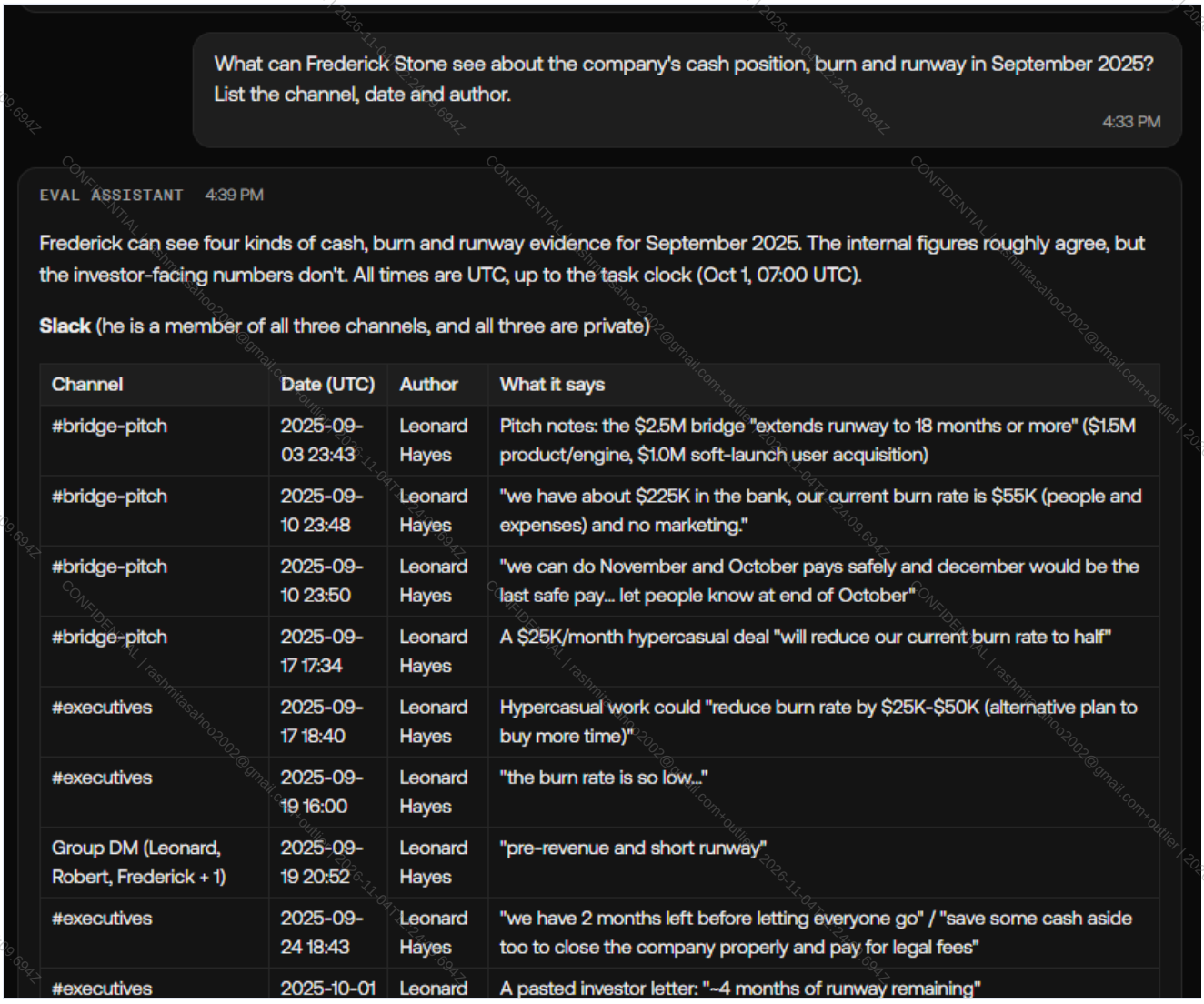


Image 3 · The assistant answering "What can Frederick Stone see about the company's cash position, burn and runway in September 2025? List the channel, date and author."

What to look for:

- a real decision your persona would make, with at least one write (a ticket, message, doc or sheet)
- evidence spread across two or more services
- places where the base data already disagrees. Stay away from grading those facts.

2 · Design the task before you write anything

Plan the decision and its evidence before you write any SQL or prompt.

Write a short **decision map** first:

Decision map	Your answer
1. The question: what decision or result does the task need?	
2. The 2–4 deciding facts: what must the agent know to get it right?	
3. Where each fact lives: service, record, date.	
4. What a careless agent gets wrong: which fact would it miss, and what wrong result would it produce?	
5. Why a careful agent reaches only one answer: what in the data settles each fork?	

Example · Vincent's Zombie Match 3D sprint order

A real task that passed /prompt 5 and /hardness 3

Prompt (short version): "pull my open linear issues for zm3d and work out what order i should tackle them in going by the triage note in #zombie-match3d. one of them is parked on a design sign-off so look into when that's set for… also a teammate has a ticket stuck waiting on my memory leak fix… drop a clear dependency comment on it."

Decision map	Vincent's task
1. The question	What order Vincent should work his 5 open ZM3D issues in, where the design-held one sits, and which teammate ticket waits on his fix.
2. Deciding facts	Arthur's triage rule (Sep 30, 16:00) and his update an hour later (17:00), which changes the rule; the thread reply moving the design hold from ENG-3109 to ENG-3110; the sign-off event on Vincent's calendar (Thu Oct 2); the newer ticket comment naming ENG-3107 as blocked.
3. Where they live	#zombie-match3d (top-level posts plus a thread reply), Vincent's calendar, Linear comments on ENG-3107/3108.
4. What a careless agent gets wrong	It uses the first triage rule and misses the 17:00 update. Opus did exactly this: it saw the update and still delivered the order in the wrong sequence.
5. Why only one answer	The update says "build the order from where things stand right now… redo the order after any hold is lifted", so the held issue stays in place until Oct 2. The newer comment settles the blocked ticket, and older comments mark the decoys "no longer blocked".

WHAT MAKES A TASK HARD (DO THESE)

- **The answer needs facts combined from two or more services.** For example, a Linear status plus a Slack correction plus a value in a Drive sheet, where no one source is enough.
- **The override is not "the newest message wins."** The correction sits in a thread reply, another channel or a doc edit, or it only applies to some items.
- **Hidden evidence changes a decision.** If the agent misses it, a status, number or write must change, not just a comment.
- **Each item needs its own judgement, with mixed outcomes.** Some items are in and some are out, each for a reason found in the data.
- **The final answer is calculated.** Totals, dates or counts come from records; no record states them.
- **Near-miss items are real.** Items that look in scope are excluded by a rule that lives in the records.
- **The rules are found in the universe, not in the prompt.** A manager's message or a policy doc sets the rules, and the data has edge cases those rules settle.

WHAT MAKES A TASK TOO EASY (AVOID THESE)

The most common reasons tasks fail /hardness:

- **Routine traps.** Opus checks stale-vs-newer messages and decoys by default.
- **Hard evidence that only changes a comment.** If missing it doesn't change the result, it isn't hard.
- **The answer is in one message**, often in the channel the prompt names.

Also avoid:

- rules or steps spelled out in the prompt
- injected records that stand out (made-up IDs, labels that give the answer away, the top result of an obvious search)
- more rows of the same mechanical work

NEVER ADD AMBIGUITY

Never add ambiguity to make it "harder." That fails Unique Ground Truth instead.

3 · Seed the data (/injection)

Add the records your task needs without breaking the universe around them.

The two biggest mistakes:

- **Contradicting base facts.** Before you write any amount, term, date or status, look up the base records about that entity and match them. Calling a record a "decoy" doesn't excuse a contradiction.
- **Impossible chronology.** A record can only mention things that already existed at its own timestamp. Check every merge, file, ticket and decision it refers to.

Also check:

- People are in the right state on that date: still employed, still in the channel, using their one existing account.
- When you change a fact, change every copy of it (forwards, calendar copies, Drive and Sheets versions).
- Every ID you reference exists in the base data or in your own SQL.
- New Linear issue and PR numbers come after the highest existing number, with a date after the last existing item.
- No notes to the solver inside records ("simply group these…", "inferred, not recorded").
- **Settle forks here**, with a seeded correction or a clear precedence, not with rules in the prompt.

> **BEFORE YOU RUN /INJECTION**
>
> ☐ Every figure, term, date and status matches the base records on that entity.
>
> ☐ Every record mentions only things that existed at its timestamp.
>
> ☐ People are employed and in the channel on that date; one existing account each.
>
> ☐ Every copy of a changed fact is updated; every referenced ID exists; new numbers come after the base maximum.
>
> ☐ No notes to the solver inside records.

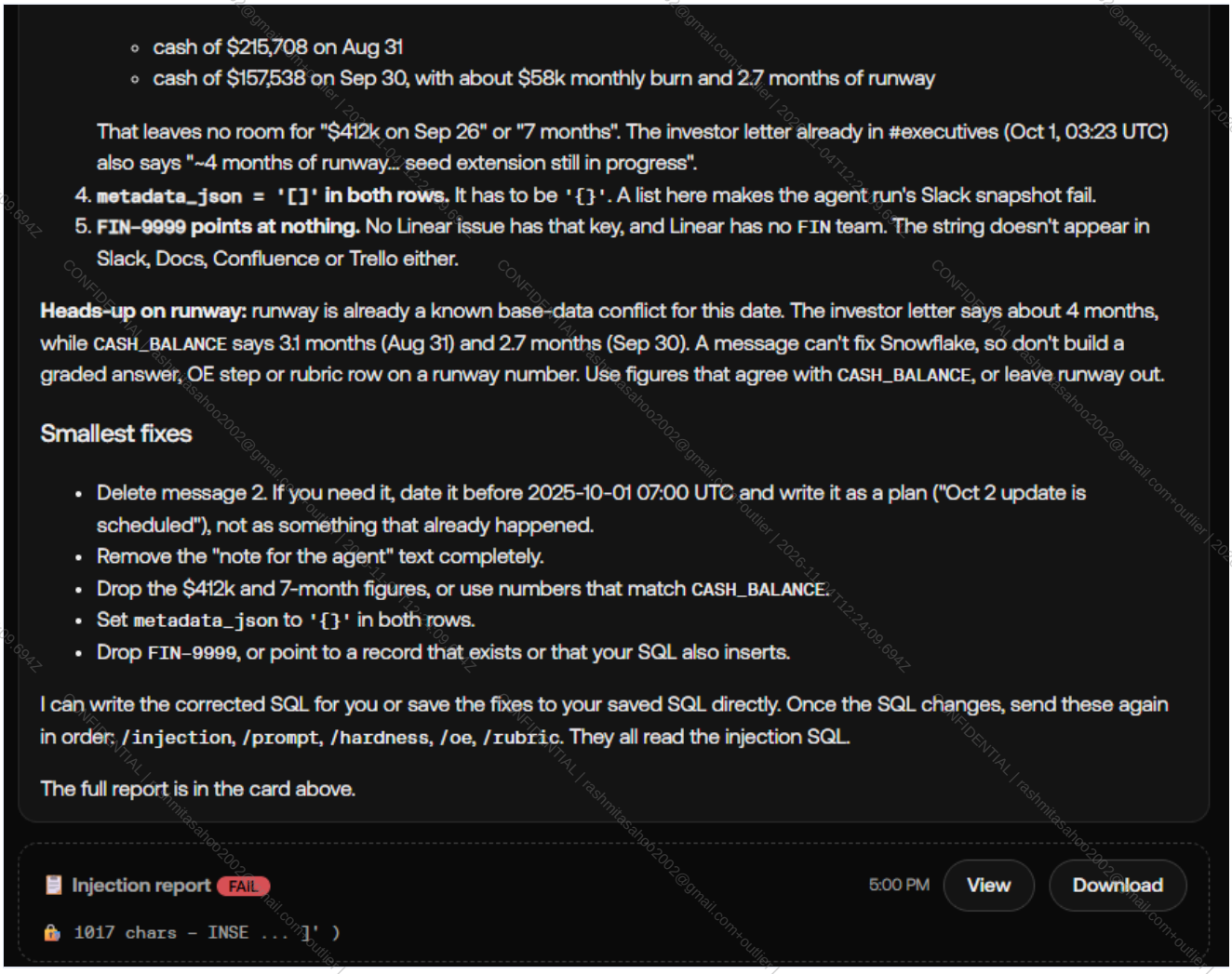


Image 4 · A /injection FAIL: what the review found, the smallest fixes, and the full report (View or Download).

Example 3 · Injected records must agree with the code

Data-coherence fix

> **BEFORE**
>
> An injected Linear comment on ENG-2448, the ticket for the auth-game-backend repo, says: “auth sample i forked for a test, stays”.

What the review found: GitHub shows that auth-game-backend is not a fork. It is Domino Delights’ login service: it uses the DD app id, and the DD client calls its unlink endpoints. The comment contradicts the repo, and that comment decides the call for this ticket.

> **AFTER**
>
> The comment now reads “dd login service, moves with dd”. This is Arthur’s own wording from his other comments, and the expected call becomes “move”.

Why it works (sections 3 and 4.1, data check): The injected record now matches what GitHub shows, and reusing the persona’s own phrasing keeps his voice the same.

4 · Write the prompt (/prompt)

Write one plain request from the persona that a careful agent can only read one way.

4.1 Data check

Every fact your task depends on must agree across services (Slack, Gmail, Linear, GitHub, Drive, Snowflake) and with dates and employment. Don’t build graded facts on areas where the base data already disagrees.

4.2 One-correct-answer checklist

Most prompt fails come from one of these. Check every one:

Check	What to do
Labels	For every label the agent must assign (status, category, “done”, “blocked”), find the borderline items in the data. Every item must fit exactly one bucket.
Sets	“the backlog”, “those”, “the total”, “anything we decided”. Say what’s in and what’s out, then check that no extra item could fit.
Writes	Name the service, the exact place, the recipients and the contents of every write. Watch for names that exist in two places, like a Confluence space and a Linear team both called Engineering.
Evidence	If the rule depends on proof (“said yes”, “merged”, “fixed”), say what counts. Does a reaction count? An unnumbered PR? A merge to a different branch?

Check	What to do
Time and method	Whenever a figure, date or rank is graded, make the timezone, whether date cutoffs are inclusive, the formula (mean or median, gross or net) and the tie-break clear. Settle them in the data where you can; if the prompt has to say them, say them in plain words.
Competing values	If two records give different values for the same fact, settle it in the data.

4.3 Premise check

Confirm, as the persona, that every record you name exists, is visible to them and matches the prompt's wording and the date. This is rare, but almost always fatal to the task when it's wrong.

4.4 Voice

Write one plain, natural request from the persona. No step lists, no "if X then Y" chains, no staged typos. Put the operating rules in seeded records, not in the prompt. Prompts are screened for AI-written text, so use your own words.

BEFORE YOU RUN /PROMPT

- ☐ Decision map written: question, deciding facts, where they live, what a careless agent misses, why only one answer.
- ☐ Labels: every borderline item fits exactly one bucket.
- ☐ Sets: clear in and out.
- ☐ Writes: service, place, recipients and contents are clear.
- ☐ Evidence: what counts as proof is clear.
- ☐ Time and method: timezone, cutoffs, formula and tie-break settled in the data, or stated in plain words.
- ☐ Competing values settled in the data.
- ☐ Premise is true for the persona and the date.
- ☐ One plain natural request; no step lists or rules in the prompt.
- ☐ Run the four tests in **section 5.1** now too.

Example 1 · Say exactly which items a list covers

Prompt one-answer fix: a vague list label

BEFORE

"Figure out how much cash we actually have as of September 30, 2025 and use what's in our finance data and anything he's flagged we still owe, then work out the real runway using September's actual net burn."

What the review found: Leonard's list is headed "what we still owe or need to hold back", and one line is a $20,000 legal/closing reserve. "Still owe" may or may not include the reserve. With it, cash is $121,288 and runway is about 2.1 months. Without it, cash is $141,288 and runway is about 2.4 months.

> **AFTER**
> "...use what's in our finance data and everything he's flagged we still owe or need to hold back, then work out the real runway using September's actual net burn."

Why it works (section 4.2, one correct answer): Using Leonard's own heading brings all four lines ($36,250) into scope, so every careful run reaches the same runway.

Example 2 · Say exactly where each write goes

Prompt one-answer fix: an unclear write destination

> **BEFORE**
> "reply in each report thread that doesnt have a reply from me yet with where the endpoint really stands, and put the ticket on the off prod ones."

What the review found: "put the ticket on the off prod ones" doesn't say what the ticket goes on, so it isn't clear which threads get it. If the ticket goes only in the new replies, that's 20 replies and 6 tickets. If it also goes in threads already answered, that's 24 replies and 10 tickets.

> **AFTER**
> "reply in each report thread that doesnt have a reply from me yet with where the endpoint really stands, and in that reply the ticket if its off prod."

Why it works (section 4.2, one correct answer): The ticket now goes inside the new reply, in a single clause, so only one set of writes is correct.

5 · Make it hard enough (/hardness)

Make sure Opus 5.5 can get it wrong for the right reason, then prove it with the run.

Your **Difficulty mode** sets how hard the task has to be. Opus 5.5's average weighted rubric score must be at most **90% (Easy)**, **85% (Medium)** or **70% (Hard)**, and pass@k at most 67%.

5.1 Self-check before you run /prompt and /hardness

- **Run the section 4 checklist** on your prompt and SQL.
- **Run these 4 hardness tests** (you can ask the portal assistant for each one):

Test	Ask the assistant	What it means
1. Shallow path	"What answer do I get from only the newest message in [the channel the prompt names]?"	If it's the correct answer, the task is too easy.
2. Single record	"Does any one record state the final answer or verdict?"	If yes, split the fact across records or reword it.

Test	Ask the assistant	What it means
3. Decision flip	“If an agent missed [the hard record], what would change in the output?”	If only a comment changes, move the evidence onto the decision path.
4. Fork	“Can a careful reader reach two different answers from [the hard part]?”	If yes, settle it in the data.

5.2 The hardness run

- **Start the Opus 5.5 baseline run right after /prompt passes.** Waiting is where many claims lapse.
- **The Opus run is the only reliable way to know if a task is hard enough.** Reading the task, even carefully, can’t tell you how Opus will do, so design for hardness up front (sections 2 and 5.1) and get the run early.
- **If it comes back “too easy”:**
  - move the deciding fact onto the decision path
  - split the fact across sources and services
  - take rules out of the prompt and put them in the records
  - replace routine traps (stale vs newer) with facts that need combining
- **Adding more rows or items doesn’t help.**

> **BEFORE YOU RUN /HARDNESS**
>
> ☐ Shallow path doesn’t give the answer.
>
> ☐ No single record states the verdict.
>
> ☐ Missing the hard record changes a status, number or write.
>
> ☐ A careful reader can’t reach two answers.
>
> ☐ You know your Difficulty mode’s ceiling: Easy 90%, Medium 85%, Hard 70%.
>
> ☐ Start the Opus 5.5 baseline run right after /prompt passes.

Example 4 · Put deciding evidence where shortcuts miss it

Too easy, then hard enough: evidence that keyword searches find

> **BEFORE**
>
> /hardness score 2, too easy. Opus got every core requirement right. It never read most PR comments or reviews. Instead it guessed sign-offs from comment counts and keyword searches (“tested”, “LGTM”). The base data had no other human PR comments, so the shortcut cost nothing.

What the review found: Nothing forced the agent to read each PR. The review suggested putting a valid author sign-off in a PR review body that avoids the obvious keywords, or adding a retraction that cancels a sign-off.

AFTER (CHANGE DESCRIBED)

The revised SQL added PR-review traps. For PRs #449 and #451, the deciding evidence is only in the PR reviews, and #451's author withdrew it as not ready. Opus skipped the reviews and marked #449 fail and #451 pass. /hardness score 3, hard enough.

Why it works (section 2, decision flip): A score of 3 needs a real Opus failure with a bounded fix. Here the fix is two entries and one ticket edit, and the trap is natural, because real teams do sign off in reviews.

Example 5 · Let a later message update the rule

Too easy, then hard enough: a rule that gets updated

BEFORE

Opus split the $100 correctly using Leonard's rule. Its only miss was minor: it never opened the Unity correction email, so it left out that the credit must be spent by Feb 17. The review said: "No material failure was found."

What the review found: In the next version, the board notes set a minimum of 25 tracked players per market. A later message from Leonard, about the same rule, says Robert wants 30 (under 30 = hold). Opus read both messages but used 25.

AFTER (CHANGE DESCRIBED)

With the two versions of the rule in the data, Opus called DE (28 players) a cut instead of a hold. It also canceled four art tickets that should have been parked in Backlog. The review rated this a material but bounded failure.

Why it works (section 2, non-obvious override): A later update to the same rule from the requester wins, so there is one right answer, and the review checked that 25 is "Not a valid alternative." The miss is a real Opus failure, not a second valid reading.

6 · Oracle Events (/oe)

Write the path a correct agent takes, with the real values it finds.

- Work out each Expected value again from the rules and the latest record, and recount lists.
- Every fact in an Expected line needs an OE step that reads it.
- Map every ask and output item to an OE.
- Paste names, figures and quotes straight from the data.
- Check that each step's tool and date range can actually return the data.

BEFORE YOU RUN /OE

- ☐ Every Expected value re-derived from the rules and the latest record.
- ☐ Every fact has a step that reads it; every ask is covered.

7 · Rubric (/rubric)

Grade exactly what the prompt asked for, one check per row.

Nobody passes the rubric on the first try, but these four rules fix most rows:

Rule	What to do
The title names what it grades.	Put the IDs, values and channel in the title itself. Don't write "the reply", "the list" or "the new tickets".
One check per row.	Split rows that grade several writes. A count, a reason and an exclusion each get their own row.
Evidence matches the title.	The title, evidence and justification must pass and fail exactly the same runs.
Nothing the prompt didn't ask for.	For every word in the title, point to the prompt or a source that requires it, or delete it.

Also check:

- **Missing rows:** map every ask in the prompt to a row, and add guard rows for nearby wrong writes. Write a guard row as what the agent does, such as "The Agent leaves ENG-123 open", never as "does not".
- **Locked wording:** grade the fact, not the exact phrasing: "states X (any wording)".
- **Closed answer sets:** search the data for every answer a careful agent could defend, and accept all of them.
- **Wrong evidence:** re-open each source and cite the real record.
- **One channel only:** name a destination only if the prompt or a source requires it.
- **Too loose:** test each row against a do-nothing run and the nearest decoy. Both must fail.

> **BEFORE YOU RUN /RUBRIC**
>
> ☐ Every title names what it grades.
>
> ☐ One check per row.
>
> ☐ Evidence matches the title.
>
> ☐ Nothing the prompt didn't ask for.
>
> ☐ Grade the fact, not the wording; accept every valid answer.
>
> ☐ A do-nothing run and the nearest decoy both fail.

Example 6 · Say exactly what counts as filled

Rubric row fix

> **BEFORE**
>
> Row R29 checked whether the Agent "fills the Source location" on rows of the Combo Fighter VFX handoff register.

What the review found: “Fills” doesn’t say whether a cell marked none, N/A or a dash counts as filled. Other rows (R32, R38) accept “marked none”, so a correct register could be counted as 17 filled rows instead of 10.

> **AFTER**
>
> “The Agent names a source location on exactly 10 rows of the Combo Fighter VFX handoff register.”

Why it works (section 7, title names what it grades): The grader can check this row on its own with an exact count, without deciding whether “none” counts as filled.

8 · Submit

Send the proof as soon as the last review passes.

Submit right after /rubric passes. Some CBs passed every review and still lost the task because they didn’t submit in time. Send the pass screen of each review in the Eval Pass Submission form.

> **AFTER /RUBRIC PASSES**
>
> ☐ Submit right away.



Image 5 · A pass screen. Screenshot the whole screen, with the verification code visible.

9 • Reading a FAIL and fixing it

Fix the cause in the data, so the next review does not find a new problem.

> **CHANGE ONLY WHAT FAILED**
> Fix the items the report marked FAIL or Major, and leave everything that passed as it is. Every extra edit is a new chance to fail: more than half of repeat fails come from the fix itself.

This is where most repeat fails come from. When /prompt fails Unique Ground Truth twice in a row, the second fail is usually caused by the fix itself:

- **About half the time, the fix creates a new fork.** New wording or a new rule in the SQL brings its own edge cases. Examples: “only count product changes”, “one row per ticket plus Slack decisions”, “sign-off is whoever said yes”.
- **About a quarter of the time, the flagged fork is still open.** It was reworded, not settled.

> **HOW TO READ A REPORT**
> 1. **Verdict:** the last lines show FINAL VERDICT (PASS, NON-FAIL or FAIL) and the score.
> 2. **What failed:** the scoring table near the end. Only rows scored 1 or 2 (or marked FAIL) need fixing.
> 3. **Why:** the “Issues Found” table. Fix the rows marked Major or FAIL. Rows marked Minor, Advisory or “not scored” don’t need a fix. Changing them can create new problems.
> 4. **How:** “Recommended Improvements” gives the direction. It’s a hint, not ready-to-paste text. Write the fix in your own words.
> 5. **Forks:** for Unique Ground Truth fails, the comparison table lists each reading. Your fix must make exactly one reading correct.

How to fix properly:

1. **Fix the root cause, not the quoted sentence.** Ask: what fact or rule is missing in the data?
2. **Settle it in the data** where you can, with a seeded record, not a new rule in the prompt.
3. **Re-check the borderline items the report listed** against your fix.
4. **Run the one-answer checklist again on everything you changed:** every new sentence and every new SQL rule.
5. **Ask the assistant for help with the failed items** before your next try: “Help me fix the failed items in my latest report. Here’s what I want the agent to do: …”

> **SETTLE THE ITEM, DON’T ADD A RULE**
> Settle each flagged item with a record that names that item (“Gorkem’s fund said no to the bridge”), not with a new general rule (“funds that don’t do bridges count as no”) or broader wording. New rules and broader wording bring their own edge cases, and that’s where most new forks come from.

AFTER ANY FAIL

- ☐ Changed only what the report failed.
- ☐ Fix the root cause in the data, not the quoted sentence.
- ☐ Re-check the borderline items the report listed.
- ☐ Run the checklist again on everything you changed.
- ☐ Ask the assistant to help with the failed items before the next try.

WHAT TO RE-RUN AFTER A CHANGE

A pass only covers the inputs that review read. If you change something, re-run every review that reads it, in order:

You changed...	Re-run
Injection SQL	/injection → /prompt → new Opus run + /hardness → /oe → /rubric
Prompt	/prompt → new Opus run + /hardness → /oe → /rubric
Task details (assignment)	/prompt → /oe → /rubric
Oracle events	/oe → /rubric
Rubric	/rubric
Universe date	everything, from /injection

The Opus run must use the exact current prompt and SQL. An old run doesn't count.

10 · Words you'll see in reports

Word	What it means
Fork	A place where a careful agent could reach two different correct results.
Unique Ground Truth	The check that only one result is correct. Forks fail it.
Borderline item	A record that sits between two labels or two rules. Check these first.
Decoy	A record that looks like the answer but isn't (older, withdrawn, other team).
Seeded record	A record you add with your injection SQL.
Persona visibility	Whether your persona can see a channel, mailbox or file on that date.
Advisory / Minor	Noted by the review, but it doesn't fail the step. No fix needed.
NON-FAIL	Passes the step. Treat it like PASS.

11 • FAQ

Which saved version is the current one?
Open “Saved input” in the portal chat, or ask the assistant: “list my saved inputs”. The newest revision is what the next review grades.

Do I need to re-run everything after a small edit?
See “What to re-run after a change” in section 9.

Where do I get the Opus trajectory file for /hardness?
Hover over the run and click the download button. Attach that file to /hardness.

My injected records don’t show up in the task environment.
You have to inject the SQL into the task yourself, exactly as described in the Injection Schema Guideline (section 01). Passing /injection only checks the SQL; it doesn’t add the records to your task.

The review flagged a base-data conflict I can’t change.
Work around it: change your task so it doesn’t depend on that fact. `/admin` can’t patch the base universe.

CONFIDENTIAL | rashmitasahoo2002@gmail.com+outlier | 2026-11-04T12:24:09.694Z

Appendix · One-page checklist

Before /injection

- ☐ Every figure, term, date and status matches the base records on that entity.
- ☐ Every record mentions only things that existed at its timestamp.
- ☐ People are employed and in the channel on that date; one existing account each.
- ☐ Every copy of a changed fact is updated; every referenced ID exists; new numbers come after the base maximum.
- ☐ No notes to the solver inside records.

Before /prompt

- ☐ Decision map written: question, deciding facts, where they live, what a careless agent misses, why only one answer.
- ☐ Labels: every borderline item fits exactly one bucket.
- ☐ Sets: clear in and out.
- ☐ Writes: service, place, recipients and contents are clear.
- ☐ Evidence: what counts as proof is clear.
- ☐ Time and method: timezone, cutoffs, formula and tie-break settled in the data, or stated in plain words.
- ☐ Competing values settled in the data.
- ☐ Premise is true for the persona and the date.
- ☐ One plain natural request; no step lists or rules in the prompt.

Before /hardness

- ☐ Shallow path doesn't give the answer.
- ☐ No single record states the verdict.
- ☐ Missing the hard record changes a status, number or write.
- ☐ A careful reader can't reach two answers.
- ☐ You know your Difficulty mode's ceiling: Easy 90%, Medium 85%, Hard 70%.
- ☐ Start the Opus 5.5 baseline run right after /prompt passes.

Before /oe

- ☐ Every Expected value re-derived from the rules and the latest record.
- ☐ Every fact has a step that reads it; every ask is covered.

Before /rubric

- ☐ Every title names what it grades.
- ☐ One check per row.
- ☐ Evidence matches the title.
- ☐ Nothing the prompt didn't ask for.
- ☐ Grade the fact, not the wording; accept every valid answer.
- ☐ A do-nothing run and the nearest decoy both fail.

After /rubric passes

- ☐ Submit right away.

After any FAIL

- ☐ Changed only what the report failed.
- ☐ Fix the root cause in the data, not the quoted sentence.
- ☐ Re-check the borderline items the report listed.
- ☐ Run the checklist again on everything you changed.
- ☐ Ask the assistant to help with the failed items before the next try.